

Partitioned Hash Join with Duplicates

Module 4: MPI Report

Alessandro Mariani 654391

1 Algorithm and MPI Design

1.1 Pipeline structure

The join algorithm is a partitioned hash join with duplicates, identical to the three-phase structure from previous modules (histogram, scatter, local join) extended to the distributed setting. Each of the N nodes participates in eight barrier-separated phases:

1. Local histogram: for each relation, a per-partition record count is computed over the local input slice.
2. Local scatter into a dest-major send buffer: records are reorganised into a contiguous buffer grouped by destination rank.
3. `MPI_Alltoall` for counts: count vectors are exchanged so each rank knows how many records it will receive from every peer.
4. `MPI_Alltoallv` for payload: the full record data is redistributed; each rank receives exactly the records belonging to its owned partitions.
5. Post-communication histogram: a second histogram pass over the received data.
6. Post-communication scatter: received records are written into the local `PartitionedRelation`.
7. Local hash join: for each owned partition p , a hash table is built on R_p and probed with S_p .
8. `MPI_Allreduce`: global join count and checksums are aggregated across all ranks.

Phases 1–2 and 5–6 together form the *redistribution* component; phase 4 is the *communication* component.

1.2 Design choices

The send buffer produced in phase 2 uses a dest-major layout: records are grouped by destination rank contiguously, so `MPI_Alltoallv` can consume the buffer directly with no additional copy before transmission.

Every timed region is preceded by an `MPI_Barrier`; the reported time is the maximum across ranks of `MPI_Wtime()`. Input generation is placed outside the timed region.

In the pure MPI version, 32 MPI processes run per node (`ntasks-per-node=32`), one per physical core, giving $32N$ total ranks for N nodes.

2 Skewed Workload

The skewed workload uses the same generation scheme as Module 3: for each record, with probability ρ one of h hot keys is selected; otherwise a key is drawn uniformly at random. The expected fraction of records falling into one hot partition is

$$\frac{\rho}{h} + \frac{1-\rho}{P},$$

so with $\rho = 0.9$, $h = 4$, and $P = 256$, each hot partition receives approximately 22.5% of all records.

3 Validation

Correctness is verified by comparing three output values produced by each configuration (`join_count`, `checksum1`, `checksum2`) against the sequential reference implementation. The checksums are 64-bit unsigned sums over the matched records; their commutativity and associativity under modular arithmetic guarantee that the result is independent of the order in which records are processed or the number of ranks involved.

Validation runs use two input sizes (medium and large), with $P = 256$ and the same fixed seed and max-key as the full campaign. All 18 checks pass ($N \in \{1, 2, 4\}$ nodes, two workloads, two implementations), confirming that the MPI redistribution preserves the join semantics regardless of process count or input size.

4 Experimental Evaluation

4.1 Setup

All experiments run on the `spmcluster`: nodes node01–node08, each with 32 physical cores, interconnected via the cluster fabric. Binaries are compiled with `mpicxx -Wall -O3 -std=c++20`. For the pure MPI version, 32 MPI processes are launched per node (`ntasks-per-node=32`), giving $32N$ total ranks at N nodes and $P/(32N)$ owned partitions per rank.

Fixed parameters for all experiments: $P = 256$, $\text{seed} = 42$, $\text{max_key} = 25 \cdot 10^6$, 3 repetitions per configuration. Median times are reported throughout. Sequential baseline: $T_{\text{seq}} = 4.81$ s (uniform), $T_{\text{seq}} = 2.80$ s (skewed).

- Strong scaling: $N_R = 5 \cdot 10^7$, $N_S = 10^8$, $N \in \{1, 2, 4, 8\}$ nodes.
- Weak scaling (uniform only): per-rank $N_R = 2 \cdot 10^6$, $N_S = 4 \cdot 10^6$, $N \in \{1, 2, 4, 8\}$ nodes.
- Skewed workload: $\rho = 0.9$, $h = 4$ hot partitions out of $P = 256$.

4.2 Strong scaling: Uniform workload

Table 1 shows the median time of each phase as the number of nodes increases.

At $N = 1$ the 32 MPI processes run entirely within shared memory, achieving 0.662 s (7.27 $\times$ over sequential). Each rank owns $P/32 = 8$ partitions and the `MPI_Alltoallv` is a fast intra-node copy (0.068 s, 10.3% of total).

As N grows, the local computation phases scale near-linearly:

- Histogram and scatter local scale at 6.9–8.0 $\times$ at $N = 8$. Each rank processes $1/(32N)$ of the

Table 1: Pure MPI per-phase times [s], uniform strong scaling ($32N$ ranks).

Phase	$N = 1$	$N = 2$	$N = 4$	$N = 8$	$S_{1 \rightarrow 8}$
Hist local	0.044	0.021	0.011	0.006	8.0 $\times$
Scatter local	0.131	0.065	0.032	0.019	6.9 $\times$
Alltoall sizes	<1 ms	11 ms	11 ms	17 ms	—
Alltoallv payload	0.068	0.287	0.807	0.544	0.13 $\times$
Hist post	0.041	0.020	0.012	0.005	7.9 $\times$
Scatter post	0.083	0.041	0.021	0.011	7.3 $\times$
Local join	0.261	0.137	0.075	0.042	6.2 $\times$
Total	0.662	0.595	0.980	0.658	1.01$\times$

total input; doubling nodes halves the per-rank workload proportionally.

- The post-communication phases (hist post, scatter post, join) scale similarly, because the owned data shrinks proportionally with ranks.
- Local join scales at 6.2 $\times$: slightly sub-linear because, while the number of partitions per rank falls from 8 (at $N = 1$) to 1 (at $N = 8$), the hash-table size per partition is fixed at N_R/P records regardless of N . The residual $8/6.2 \approx 1.3\times$ gap comes from per-rank fixed costs—barrier synchronisation, hash-table allocation—that are amortised over fewer partitions at higher N .

The `MPI_Alltoallv` collective, however, becomes the dominant bottleneck. The implementation uses a pairwise exchange: each of the $32N$ ranks performs $32N$ sequential `MPI_Sendrecv` steps, one per peer, keeping at most one message pair in flight at a time. For strong scaling the total data volume is fixed, so each step carries $\mathcal{O}(1/N)$ of the data; bandwidth cost stays roughly constant but the latency overhead grows as $\mathcal{O}(N)$. Going from $N = 1$ (32 steps, all shared-memory) to $N = 4$ (128 steps, mostly inter-node) raises the Alltoallv from 0.068 s to 0.807 s. At $N = 4$ it represents 82.4% of total time. At $N = 8$ (256 steps) the Alltoallv falls to 0.544 s as each step carries less data, accounting for 82.6% of total time. The resulting scaling pattern is non-monotone: total time improves from 0.662 s ($N = 1$) to 0.595 s ($N = 2$, best at 8.09 $\times$ over sequential), then rises to 0.980 s ($N = 4$) as inter-node Alltoallv latency dominates, before recovering to 0.658 s at $N = 8$. The overall $S_{1 \rightarrow 8} = 1.01\times$ shows that 8-node pure MPI delivers

Table 2: Pure MPI per-phase times [s], skewed strong scaling (32N ranks).

Phase	$N = 1$	$N = 2$	$N = 4$	$N = 8$	$S_{1 \rightarrow 8}$
Hist local	0.044	0.022	0.011	0.006	$8.0\times$
Scatter local	0.106	0.053	0.026	0.014	$7.7\times$
Alltoall sizes	<1 ms	12 ms	9 ms	16 ms	—
Alltoallv payload	0.112	0.397	0.557	0.852	$0.13\times$
Hist post	0.199	0.188	0.188	0.189	$1.05\times$
Scatter post	0.283	0.268	0.263	0.264	$1.07\times$
Local join	0.684	0.674	0.670	0.669	$1.02\times$
Total	1.471	1.657	1.764	2.048	$0.72\times$

essentially the same throughput as single-node; the best MPI configuration is $N = 2$.

The Alltoall sizes collective grows from <1 ms to 17 ms as N increases but remains negligible (<3% of total) throughout.

4.3 Strong scaling: Skewed workload

Table 2 reports the per-phase breakdown for the skewed workload. The pre-communication phases (hist local and scatter local) scale as well as for the uniform case: each rank processes the same total number of records regardless of their key distribution.

The three post-communication phases (hist post, scatter post, join) show essentially *no scaling* ($\approx 1.05\times$ at $N = 8$), due to two effects:

1. *Load imbalance from fixed ownership.* With $32N$ ranks each owning one or a few partitions, the $h = 4$ hot partitions are always assigned to 4 fixed ranks. Each hot-partition owner receives $\approx 22.5\%$ of the global N_R , regardless of N . Since MPI requires all ranks to synchronise at the Allreduce, the total time is governed by the *slowest rank*. Adding more nodes does not reduce the hot owner’s load.
2. *Growing Alltoallv with asymmetric loads.* Skew creates severely asymmetric message sizes: hot-partition-owning ranks absorb far more data than cold ones. Even at $N = 1$, where all communication is intra-node shared-memory, this asymmetry serialises the pairwise exchange: hot-partition owners must receive large blocks from every peer, making the skewed Alltoallv (0.112s) already $1.65\times$ slower than the uniform intra-node case

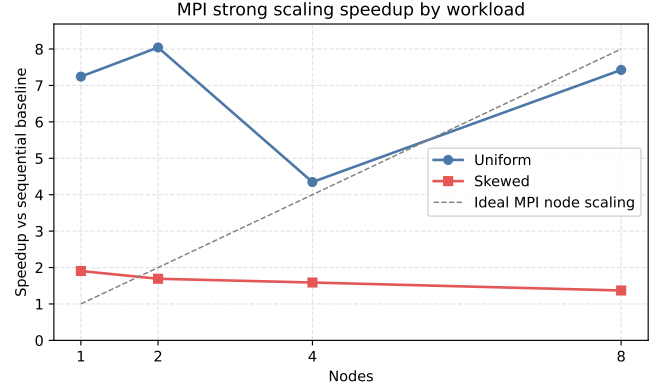


Figure 1: Pure MPI strong-scaling speedup over the sequential baseline.

Table 3: Pure MPI strong-scaling summary.

N	Uniform ($T_{\text{seq}} = 4.81$ s)			Skewed ($T_{\text{seq}} = 2.80$ s)		
	T [s]	S_N	vs. seq	T [s]	S_N	vs. seq
1	0.662	1.00	$7.27\times$	1.471	1.00	$1.91\times$
2	0.595	1.11	$8.09\times$	1.657	0.89	$1.69\times$
4	0.980	0.68	$4.91\times$	1.764	0.83	$1.59\times$
8	0.658	1.01	$7.31\times$	2.048	0.72	$1.37\times$

(0.068 s) despite identical total data volume. The imbalance worsens as N grows and inter-node transfers amplify the serialisation, causing the Alltoallv to increase monotonically to 0.852 s at $N = 8$.

Total time increases from 1.471 s ($N = 1$) to 1.764 s ($N = 4$) and 2.048 s at $N = 8$, yielding a self-speedup of $0.72\times$. At all node counts, total time remains below the sequential baseline (2.80 s); at $N = 1$, the 32 intra-node processes achieve $1.91\times$ over sequential.

Figure 1 and Table 3 summarise the overall strong-scaling results.

4.4 Weak scaling: Uniform workload

Figure 2 and Table 4 show the MPI weak-scaling results on the uniform workload, where the per-rank input is held fixed at $N_R^{\text{rank}} = 2 \cdot 10^6$, $N_S^{\text{rank}} = 4 \cdot 10^6$ (total input grows as $32N \cdot 2 \cdot 10^6$).

At $N = 1$ (32 processes) the total time is 0.829 s. By $N = 2$ (64 processes) it increases to 1.433 s, dropping efficiency to 57.9%. By $N = 4$ (128 processes)

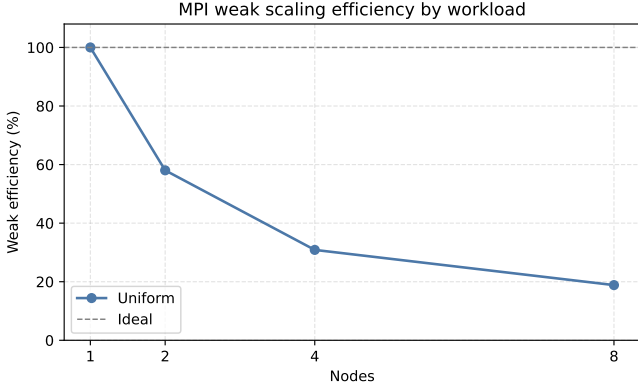


Figure 2: Pure MPI weak-scaling efficiency (uniform workload).

it reaches 2.750 s (30.2% efficiency) and degrades further at $N = 8$ (4.347 s, 19.1%).

The efficiency loss is attributable to the pairwise Alltoallv: with $32N$ total ranks, the exchange performs $32N$ sequential `MPI_Sendrecv` rounds. In weak scaling the per-rank input is fixed, so total data grows as $\mathcal{O}(N)$ and the per-round message size stays approximately constant. Total communication time is therefore $\mathcal{O}(N \times \text{latency} + N \times \text{bandwidth})$, i.e. linear in N . Going from $N = 1$ to $N = 8$ multiplies the round count from 32 to 256 and shifts the majority of rounds from shared-memory to inter-node paths, making Alltoallv the dominant cost at $N \geq 2$.

4.5 Weak scaling: Skewed workload

Skewed weak-scaling experiments were not performed, as the strong-scaling results already demonstrate that adding nodes under skewed data degrades both the Alltoallv (due to asymmetric message sizes) and the post-communication phases (due to fixed hot-partition ownership). These effects would be amplified in a weak-scaling regime as the absolute amount of hot-partition data grows with N , making the bottleneck progressively worse.

4.6 Phase breakdown

Figure 3 decomposes the total time into three logical components:

- Redistribution: phases 1+2+5+6 (local histogram and scatter, both pre- and post-Alltoallv).

Table 4: Pure MPI weak-scaling time and efficiency (uniform).

N	T [s]	η [%]
1	0.829	100.0
2	1.433	57.9
4	2.750	30.2
8	4.347	19.1

Table 5: Alltoallv (pure MPI) fraction of total time.

N	Pure MPI [%]
1	10.3
2	48.2
4	82.4
8	82.6

- Communication: phases 3+4 (Alltoall sizes + Alltoallv payload).
- Local join: phase 7.

On the uniform workload (Figure 3, left), redistribution and join together dominate at $N = 1$ (45.2% and 39.4% respectively), with communication a minor contributor (10.3%, 0.068 s). As N grows, redistribution and join shrink near-linearly while communication rises. At $N = 8$, the Alltoallv payload alone consumes 82.6% of total time (Table 5); adding the Alltoall-sizes phase (17 ms, 2.6%) brings total communication to 85.3%, leaving redistribution (6.2%) and join (6.4%) as small components. Communication is the dominant, non-scalable floor of the pure-MPI pipeline.

On the skewed workload (Figure 3, right), the phase distribution differs at all node counts. At $N = 1$, redistribution (43.0%) and join (46.5%) account for the majority of total time while communication is 7.6%, reflecting the cost of processing hot-partition data in the post-communication phases. As N grows, communication rises to 41.6% at $N = 8$, while the post-communication phases remain approximately constant ($\approx 55\%$) due to load imbalance. Both effects cause total time to increase with N .

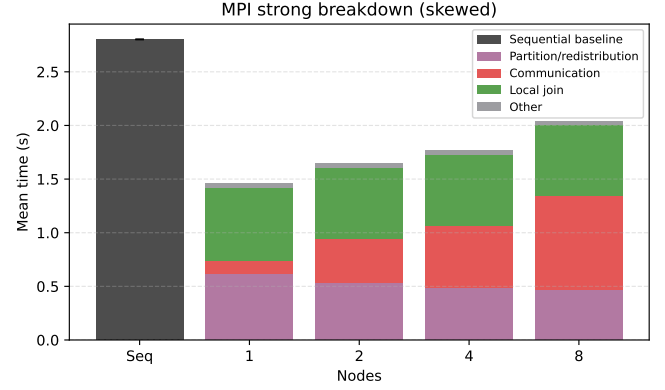
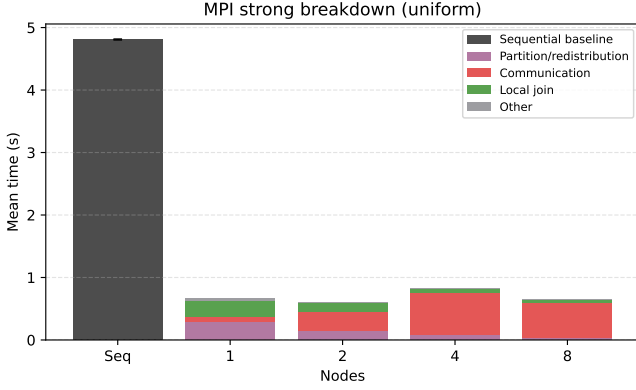


Figure 3: Pure MPI phase breakdown, uniform and skewed workloads.

5 Discussion and Conclusions

Three distinct bottleneck categories characterise the distributed hash-join pipeline.

Pre-communication phases are near-perfectly parallel: histogram and scatter scale at 6.9–8.0 $\times$ at $N = 8$ on the uniform workload. These phases are purely data-parallel and local, processing $1/(32N)$ of the total input per rank.

Communication is the dominant non-scalable component (Section 4.6). With 32 MPI processes per node, the pairwise exchange performs $32N$ sequential rounds whose latency grows $\mathcal{O}(N)$ with nodes: Altogether grows from 10.3% at $N = 1$ to 82.6% at $N = 8$ (uniform), exceeding all compute phases combined.

Post-communication phases are limited by load imbalance on skewed data (Section 4.3): at $N = 8$, these phases scale at only 1.02–1.07 $\times$ instead of the 7–8 $\times$ observed on uniform data.

With 32 processes per node, the pure MPI version achieves 7.27 $\times$ over sequential at $N = 1$. Increasing N raises the pairwise round count, increasing total time at $N = 4$ (0.980s) before it decreases at $N = 8$ (0.658s) as per-round data volume falls.

6 Comparison with previous modules

Table 6: Cross-module comparison, uniform workload ($T_{\text{seq}} = 4.81$ s).

Implementation	Time [s]	Speedup
Sequential	4.81	1.00 $\times$
M2 C++ threads (32t)	0.66	7.29 $\times$
M3 OpenMP loop (32t)	0.65	7.44 $\times$
M4 MPI (1 node, 32p)	0.66	7.27 $\times$
M4 MPI (2 nodes, 64p)	0.60	8.09 $\times$
M4 MPI (8 nodes, 256p)	0.66	7.31 $\times$

6.1 Uniform workload

Table 6 compares all Module 4 MPI configurations against the sequential baseline and the single-node shared-memory implementations from Modules 2 and 3.

The pure MPI version at $N = 1$ (32 processes) achieves 7.27 $\times$ over sequential, comparable to Module 3 OpenMP (32 threads, 7.44 $\times$) and Module 2 C++ threads (32 threads, 7.29 $\times$): all three exploit 32 physical cores on one node, and their runtimes converge at this level of intra-node parallelism.

Scaling pure MPI to $N = 2$ yields the best MPI result at 8.09 $\times$ (0.595s). At $N = 8$ (256 processes) the result is 7.31 $\times$ (0.658s), essentially matching the single-node result. The non-monotone path ($N = 4$ peak at 0.980s, 4.91 $\times$) is driven by the $\mathcal{O}(N)$ latency growth of the pairwise exchange.

Table 7: Cross-module comparison, skewed workload ($T_{\text{seq}} = 2.80$ s).

Implementation	Time [s]	Speedup
Sequential	2.80	1.00×
M3 OpenMP loop (16t)	0.27	10.27×
M4 MPI (1 node, 32p)	1.47	1.91×
M4 MPI (8 nodes, 256p)	2.05	1.37×

6.2 Skewed workload

Table 7 shows the skewed-workload comparison.

The pure MPI version at $N = 1$ achieves only $1.91\times$ over the sequential baseline and *degrades* to $1.37\times$ at $N = 8$. For reference, Module 3’s OpenMP loop at 16 threads reaches $10.27\times$ on the same skewed dataset.

Two effects account for this difference, both visible in the phase breakdown. First, `MPI_Alltoallv` redistributes every record regardless of skew, so communication cost does not benefit from fewer join matches. Second, the deterministic partition ownership ties hot partitions to fixed ranks; shared-memory threads in Modules 2/3 can dynamically share the hot-partition work, while distributed MPI processes cannot without explicit load balancing.

A MPI+OpenMP Hybrid Version

A.1 Strategy

The hybrid binary (`hashjoin_mpi_omp`) keeps the same eight-phase MPI pipeline, but parallelises the local phases with OpenMP. Histogram (phases 1, 5), scatter (phases 2, 6), and local join (phase 7) use the loop-level strategy from Module 3: static scheduling for histogram and scatter, dynamic scheduling for join. The MPI collectives are unchanged.

Each node runs *one* MPI rank with 32 OpenMP threads (`OMP_NUM_THREADS=32`, close binding), giving N total MPI ranks for N nodes. This keeps the all-to-all collective to only N^2 pairs (vs. $(32N)^2$ for pure MPI), reducing communication overhead proportionally.

Table 8: Hybrid per-phase times [s], uniform strong scaling.

Phase	$N = 1$	$N = 2$	$N = 4$	$N = 8$	$S_{1 \rightarrow 8}$
Hist local	0.069	0.033	0.019	0.011	$6.6\times$
Scatter local	0.488	0.244	0.121	0.061	$8.0\times$
Alltoallv payload	0.004	0.359	0.317	0.184	$0.02\times$
Hist post	0.042	0.023	0.020	0.007	$6.0\times$
Scatter post	0.432	0.218	0.115	0.056	$7.8\times$
Local join	0.273	0.151	0.088	0.056	$4.8\times$
Total	1.639	1.198	0.759	0.422	$3.9\times$

A.2 Experimental Evaluation

All hybrid experiments reuse the setup from Section 4.1 and the skewed-workload generation from Section 2.

Table 8 reports the hybrid strong-scaling phase times on uniform data.

Compared with pure MPI, the $N = 1$ local phases are slower: scatter rises from 0.131 s (pure MPI, 32 parallel processes) to 0.488 s (1 MPI rank with 32 OpenMP threads). At $N = 1$, the Alltoallv is near-zero (0.004 s): with a single MPI rank, all data is already local and the collective is bypassed via a direct buffer move, avoiding any MPI stack overhead. At $N > 1$, reducing the rank count to N limits the pairwise exchange to N sequential rounds (vs. $32N$ for pure MPI), bounding the collective cost. Alltoallv rises from near-zero at $N = 1$ to 0.359 s at $N = 2$ (first cross-network traffic), then decreases to 0.317 s ($N = 4$) and 0.184 s ($N = 8$) as data per round shrinks. The Alltoallv fraction reaches 43.7% at $N = 8$ (Table 5), compared to 82.6% for pure MPI.

The hybrid self-speedup at $N = 8$ is $3.9\times$ over the single-node result, and the speedup over sequential is $11.40\times$, the highest across all configurations. In contrast to pure MPI, total time decreases monotonically with N .

Table 9 shows the hybrid strong-scaling breakdown on the skewed workload.

On skewed data, the Alltoallv at $N = 1$ is near-zero (0.004 s, bypass active) and rises to 0.426–0.503 s at $N \geq 2$, remaining roughly flat across node counts: with only N ranks and $h = 4$ fixed hot partitions, some ranks always receive disproportion-

Table 9: Hybrid per-phase times [s], skewed strong scaling.

Phase	$N = 1$	$N = 2$	$N = 4$	$N = 8$	$S_{1 \rightarrow 8}$
Hist local	0.065	0.039	0.016	0.009	$7.3\times$
Scatter local	0.448	0.232	0.113	0.058	$7.8\times$
Alltoally payload	0.004	0.463	0.503	0.426	$0.01\times$
Hist post	0.041	0.028	0.036	0.017	$2.4\times$
Scatter post	0.431	0.282	0.241	0.168	$2.6\times$
Local join	0.393	0.367	0.369	0.358	$1.10\times$
Total	1.715	1.627	1.463	1.159	$1.48\times$

Table 10: Hybrid weak-scaling time and efficiency (uniform).

N	T [s]	η [%]
1	0.070	100.0
2	0.109	64.6
4	0.132	53.3
8	0.147	47.9

ately large messages regardless of N . The post-communication phases (hist post, scatter post, join) scale at only $2.4\times$, $2.6\times$, and $1.10\times$ respectively, driven by the same fixed hot-owner imbalance as pure MPI. OpenMP threads accelerate work *within* each rank but cannot redistribute hot partitions *between* ranks. The resulting self-speedup of $1.48\times$ is substantially lower than the $3.9\times$ achieved on uniform data.

Table 10 shows the hybrid weak-scaling efficiency on the uniform workload.

The hybrid achieves 47.9% weak-scaling efficiency at $N = 8$, compared to 19.1% for pure MPI. The difference is attributable to collective size: the hybrid pairwise exchange performs only N sequential rounds at $N = 8$, while pure MPI performs $32N = 256$ rounds. Local computation scales near-perfectly (scatter is fully data-parallel), so the efficiency loss comes mainly from the growing Alltoally fraction as the absolute communication cost increases with N .

A.3 Conclusions

On the uniform workload, the MPI+OpenMP configuration provides better multi-node scalability than pure MPI: limiting ranks to N reduces the pairwise round count from $32N$ to N , yielding $11.40\times$

Table 11: Cross-module comparison, MPI+OpenMP hybrid, uniform workload.

Implementation	Time [s]	Speedup
Sequential	4.81	$1.00\times$
M2 C++ threads (32t)	0.66	$7.29\times$
M3 OpenMP loop (32t)	0.65	$7.44\times$
M4 MPI+OMP (1 node)	1.64	$2.94\times$
M4 MPI+OMP (8 nodes)	0.42	$11.40\times$

Table 12: Cross-module comparison, MPI+OpenMP hybrid, skewed workload.

Implementation	Time [s]	Speedup
Sequential	2.80	$1.00\times$
M3 OpenMP loop (16t)	0.27	$10.27\times$
M4 MPI+OMP (1 node)	1.72	$1.63\times$
M4 MPI+OMP (8 nodes)	1.16	$2.42\times$

speedup at $N = 8$, the highest across all configurations. Single-node performance ($2.94\times$, 1.64 s) is lower than both the pure-MPI and shared-memory baselines; the hybrid exceeds both from $N \geq 4$ onwards.

For skewed data, the remaining bottleneck is algorithmic. Neither pure MPI nor the hybrid can move hot-partition work across rank boundaries; improving this case requires a repartitioning step or dynamic load balancing at the MPI level.

A.4 Comparison with previous modules

Tables 11 and 12 and Figures 6 and 7 compare the hybrid against the sequential baseline and the single-node shared-memory implementations from Modules 2 and 3.

On the uniform workload (Table 11), the hybrid at $N = 1$ (1.64 s, $2.94\times$) is slower than all single-node implementations: a single MPI rank uses one thread pool for scatter and histogram, incurring more synchronisation overhead than 32 independent MPI processes. At $N = 8$, the hybrid achieves 0.42 s ($11.40\times$), the highest speedup across all configurations, because reducing MPI ranks from $32N$ to N limits the Alltoally exchange to N sequential rounds.

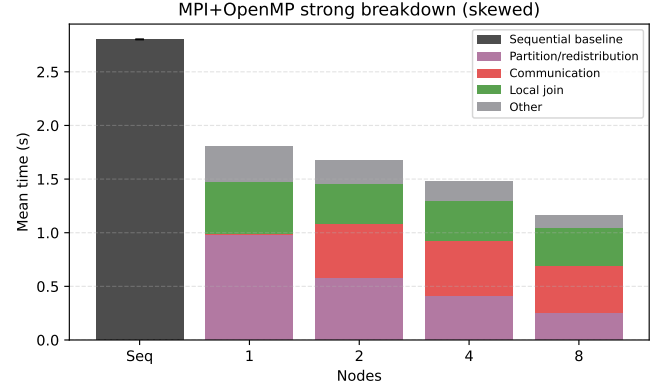
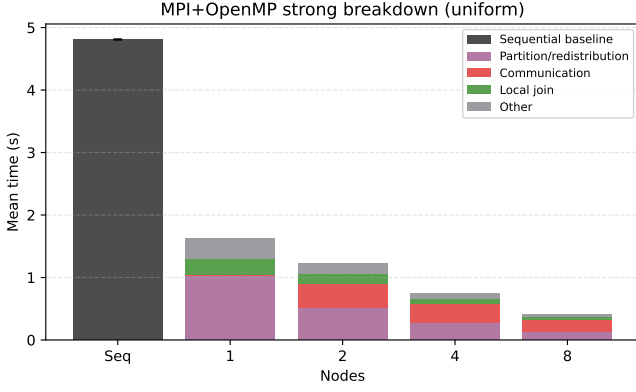


Figure 4: Hybrid phase breakdown, uniform and skewed workloads.

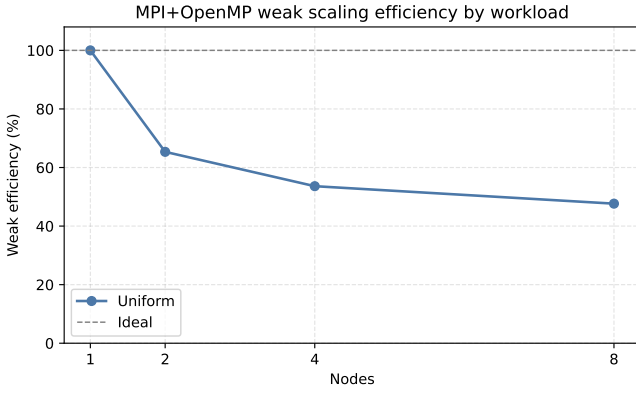


Figure 5: Hybrid weak-scaling efficiency (uniform workload).

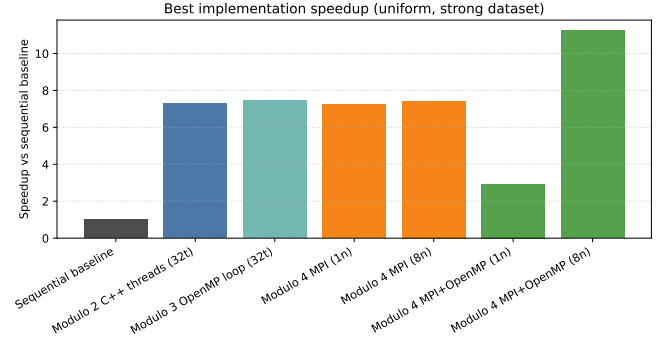


Figure 6: Cross-module speedup comparison, uniform workload.

On the skewed workload (Table 12), the hybrid improves modestly to $2.42\times$ at $N = 8$. The gap versus Module 3’s OpenMP at 16 threads ($10.27\times$) comes from the same two effects visible in the phase breakdown: `MPI_Alltoallv` redistributes every record regardless of skew, and the deterministic partition ownership ties hot partitions to fixed ranks; shared-memory threads in Modules 2/3 can dynamically share the hot-partition work, while distributed MPI processes cannot without explicit load balancing.

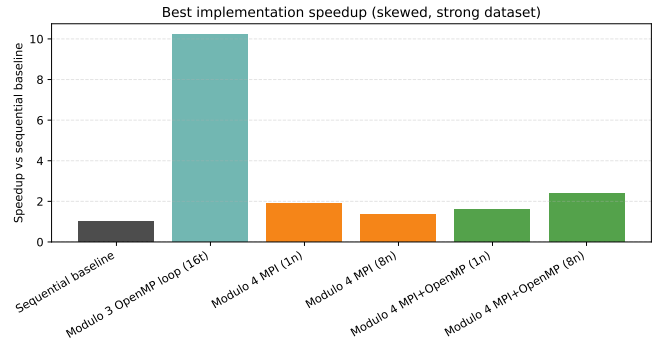


Figure 7: Cross-module speedup comparison, skewed workload.